Docker

컨테이너로 옮기기

이미지 · 레이어부터 클라우드 배포까지 | Software Engineering

최성철 교수
Director of TEAMLAB

방금 시험을 봤습니다

오늘 남은 시간

1. 쪽지시험 2 리뷰 — 개념을 다시 훑습니다
2. 실습 — 내 프로젝트를 컨테이너로 띄웁니다

지난 시간엔 Git
오늘은 환경입니다

Docker

명령어는 거의 안 나옵니다

무엇이 왜 그렇게 동작하는지만 봅니다. 시험에도 안 나왔죠

오늘의 차례

1 · 2　왜 컨테이너인가 · 격리의 세 세대

3 · 4　이미지와 레이어 · 컨테이너가 사는 세계

5 · 6　Dockerfile과 빌드 캐시 · 레지스트리와 태그

7 · 8　Compose · 클라우드에서 왜 중요한가

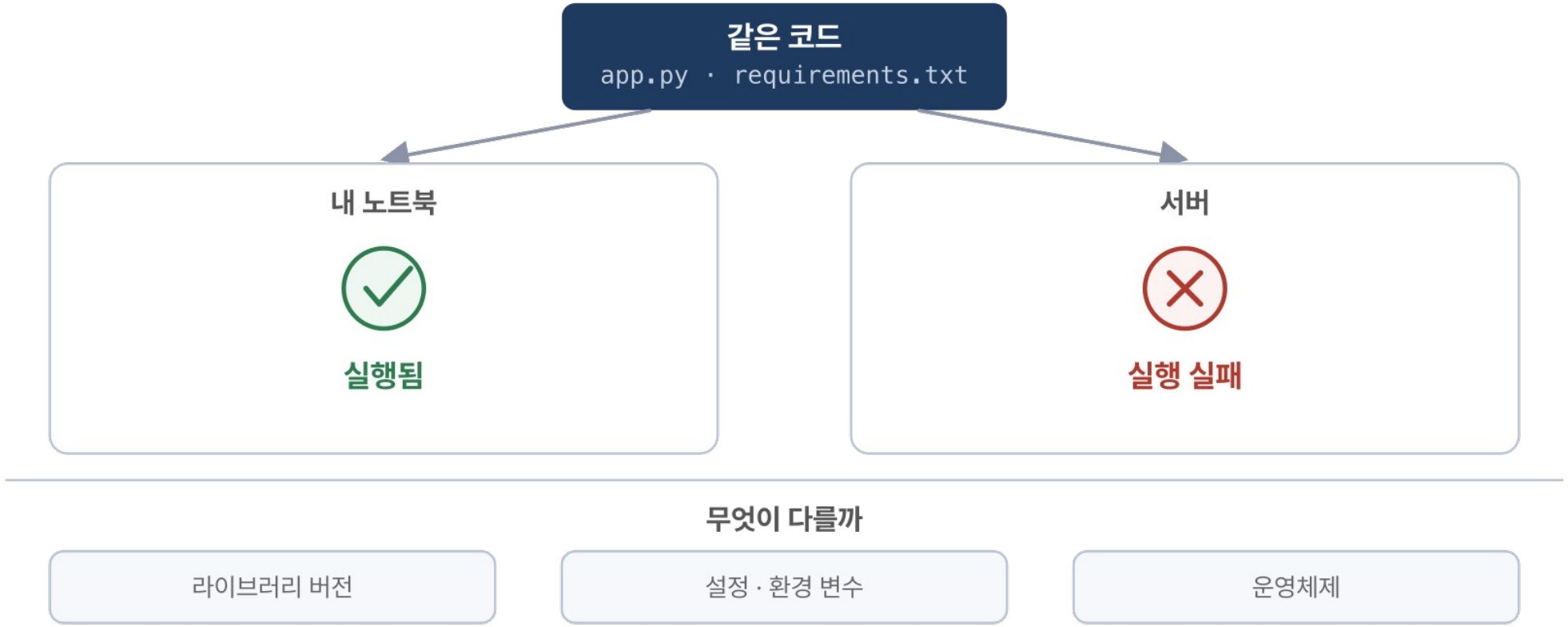
9 · 10　실무의 약속들 · 이 수업에서 쓰는 방식

1. 왜 컨테이너인가

“제 컴퓨터에서는 되는데요”

같은 저장소, 같은 커밋인데 팀원 노트북에서는 실행이 안 됩니다

같은 코드, 다른 환경



코드는 Git이 맞춰 주지만 환경은 맞춰 주지 않는다.

Git은 이걸 못 풉니다

Git이 관리하는 것은 코드이고, 문제가 되는 것은 코드가 실행되는 환경입니다

환경 차이 세 가지

1. 버전이 다르다 — 나는 Python 3.12, 팀원은 3.9
2. 설정이 다르다 — 환경 변수 · 포트 · 키 파일의 위치
3. 운영체제가 다르다 — 경로 · 줄바꿈 · 설치 방법

① 문서로 적어 둔다

모든 팀이 이렇게 시작하고, 모든 팀에서 이 문서는 낡습니다

② 컴퓨터를 통째로 준다

가상머신 이미지. 확실하지만 수십 기가바이트에 켜는 데 몇 분입니다

Docker는
이 둘의 중간을 노립니다

설치 과정을 글로 적되, 사람이 아니라 기계가 실행해 결과를 통째로 묶습니다

Git이 코드를 재현한다면
Docker는 실행 환경을 재현합니다

2. 격리의 세 세대

서버 한 대에 서비스 둘

A는 Python 3.9, B는 3.12. 둘 다 8080 포트를 쓰고 싶어 합니다

그냥 한 대에 올리면

라이브러리와 설정이 서로 부딪힌다

포트가 겹친다

A를 고치려고 재부팅하면 B도 같이 멈춘다

한쪽이 메모리를 다 쓰면 다른 쪽이 죽는다

물리 서버 · 가상머신 · 컨테이너



부팅 · 이미 켜짐
용량 · OS 전체
격리 · 없음

부팅 · 분 단위
용량 · GB 단위~
격리 · 강함 (OS별도)

부팅 · 초 단위
용량 · 수십 MB~
격리 · 프로세스 수준

게스트 운영체제가 사라진 것이 핵심이다.

2세대 — 가상머신

하이퍼바이저가 자원을 쪼개고

각 방은 자기 운영체제를 올립니다. 격리는 확실하지만 방마다 OS 한 벌이 듭니다

3세대 — 컨테이너

운영체제를
따로 줘야 할까?

커널은 같이 쓰고 나머지만 분리 — 그래서 컨테이너에는 게스트 OS가 없습니다

호스트에서 보면 프로세스 하나

다만 자기만의 파일시스템 · 네트워크 · 프로세스 번호만 보도록 가려져 있습니다

컨테이너는 가상 컴퓨터가 아니라
격리된 프로세스입니다

그 결과가 이렇게 달라진다

	가상머신	컨테이너
운영체제	게스트 OS를 따로 올린다	호스트 커널을 같이 쓴다
크기	보통 기가바이트 단위	보통 수십 ~ 수백 MB
시작	부팅이 필요해 수십 초 ~ 분	프로세스 실행이라 1초 안팎
격리	더 강하다	커널을 공유하는 만큼 덜하다

첫 줄이 원인이고 아래 세 줄은 결과다. 이 인과를 한 문장으로 쓸 수 있어야 한다.

가상머신은 이제 안 쓰나?

아닙니다. 커널을 공유한다는 것은 커널이 다르면 못 쓴다는 뜻이기도 합니다

맥과 윈도우에는 리눅스 커널이 없습니다

Docker Desktop이 뒤에서 작은 리눅스 가상머신을 켜 둡니다

3. 이미지와 레이어

이미지는
설치가 끝난 상태의 사진입니다

OS 기본 파일, 런타임, 라이브러리, 내 코드까지 — 읽기 전용입니다

흔한 오해

“이미지는 설계도”

절반만 맞습니다. 설계도는 아직 아무것도 지어지지 않은 것입니다

이미지는 이미 지어져 있습니다

설계도에 해당하는 것은 5장의 Dockerfile입니다

레이어

이미지는 통짜 파일이 아닙니다

레이어가 층층이 쌓인 구조입니다

보통 이 순서로 쌓인다

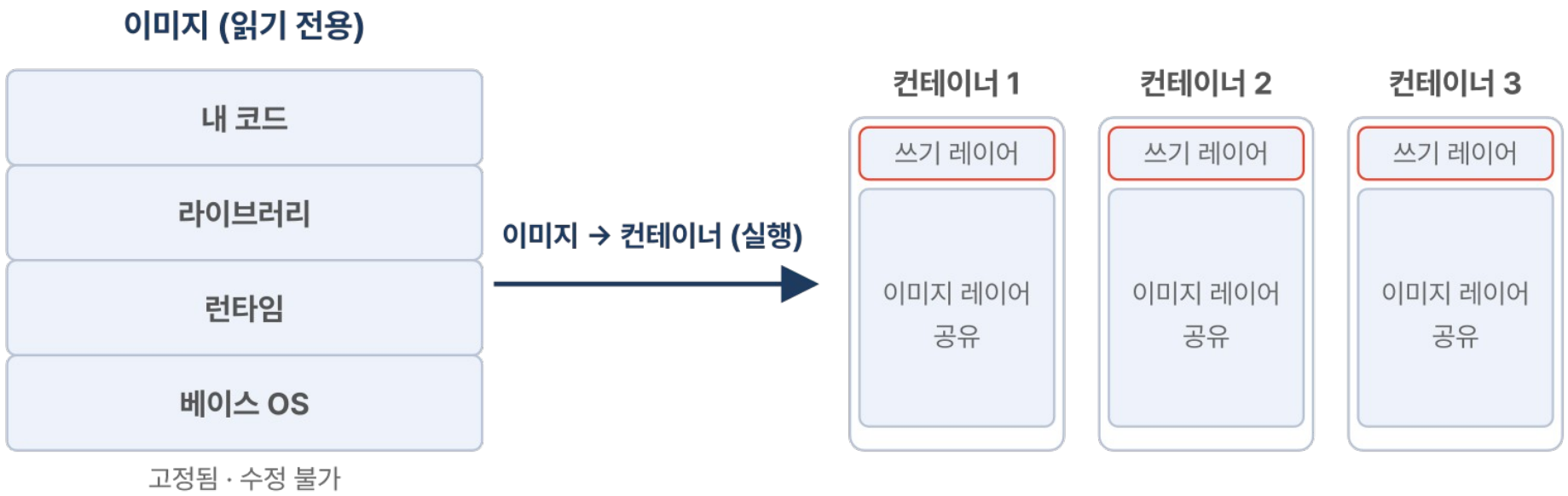
① 베이스 이미지 — 리눅스 배포판의 최소 파일들

② 런타임 — 예를 들어 Python 이나 Node.js

③ 라이브러리 — 프로젝트가 쓰는 패키지들

④ 내 코드

이미지와 컨테이너



이미지는 그대로, 컨테이너마다 변경 사항만 쓰기 레이어에 쌓인다

이미지는 읽기 전용 레이어의 스택, 컨테이너는 그 위에 쓰기 레이어를 얹은 것.

층으로 나눠 두면 좋은 점 둘

① 나눠 쓴다 — 같은 베이스와 런타임은 한 번만 받는다

② 건너뛴다 — 내 코드만 바뀌면 맨 위 층만 새로 만든다

컨테이너는 이미지에
얇은 쓰기 레이어를 얹은 것

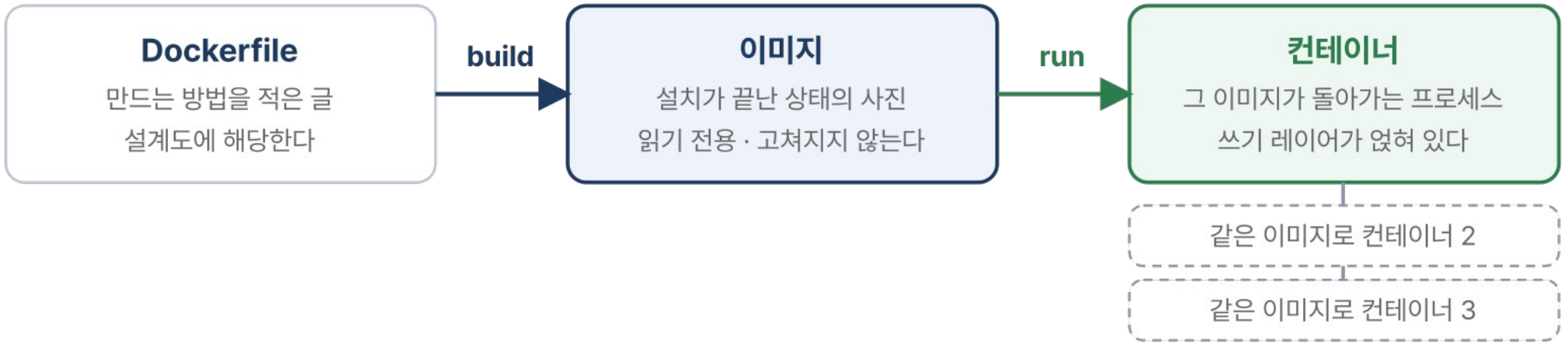
여기서 두 가지가 따라 나온다

① 같은 이미지로 여러 개 — 쓰기 레이어가 각각이다

② 지우면 데이터가 사라진다 — 쓰기 레이어도 함께

이미지는 저장된 것
컨테이너는 돌아가는 것

Dockerfile · 이미지 · 컨테이너



한 번 적는다 **한 번 만든다** **여러 번 띄운다**

build 는 이미지를 만들고, run 은 그 이미지로 컨테이너를 띄운다

build 는 이미지를 만들고, run 은 그 이미지로 컨테이너를 띄운다.

이름표 — 태그와 다이제스트

nginx:1.27

이름 : 태그

옮겨 붙일 수 있는 이름표 — 어제의 latest 와 오늘의 latest 는 다를 수 있습니다

sha256:9f3c1e...

내용으로 계산한 지문

1바이트만 달라도 값이 달라집니다. Git 커밋 해시와 같은 발상

태그와 다이제스트

태그

사람이 붙인다
옮겨 붙일 수 있다
읽기 쉽다

다이제스트

내용이 정한다
바뀌지 않는다
믿을 수 있다

“같은 태그인데 다른 이미지” 는 있어도, “같은 다이제스트인데 다른 이미지” 는 없다.

잠깐, 직접 쳐 봅시다

외울 필요는 없습니다
읽고 따라 칠 수는 있어야 합니다

실제로는 AI에게 시키게 됩니다

그래도 무엇이 벌어지는지는 화면에서 본 적이 있어야 합니다

그리고 시험에는

명령어가 나오지 않습니다

`docker run hello-world`

잘 깔렸는지 확인한다

받아서 → 컨테이너를 만들고 → 실행하고 → 끝납니다

실제로 치면

```
$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
 3. The Docker daemon created a new container from that image ...
 4. The Docker daemon streamed that output to the Docker client ...
```

이 네 줄이 오늘 배운 순서 그대로다. 받는다 → 만든다 → 실행한다. (뒷부분 줄임)

`docker pull nginx:1.27`

이미지만 받아 둔다

실행은 하지 않습니다. latest 말고 태그를 박아서 받습니다

받아 보면

```
$ docker pull nginx:1.27
1.27: Pulling from library/nginx
34ef2a75627f: Pull complete
06cdcf65323d: Pull complete
ca26eebd8366: Pull complete
27fe6a09bce8: Pull complete
447b5a3f15c6: Pull complete
9d29d0ca23f3: Pull complete
71eccce642b8: Pull complete
Digest: sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
Status: Downloaded newer image for nginx:1.27
```

줄줄이 내려오는 저 일곱 줄이 레이어다. 그리고 마지막에 다이제스트가 찍힌다.

내가 가진 이미지

```
$ docker images nginx:1.27
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:1.27     6784fb0834aa       284MB           72MB   U
```

CONTENT SIZE 는 받아 오는 크기, DISK USAGE 는 풀어 놓고 차지하는 크기다.

지금 돌아가는 컨테이너

```
$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAMES     IMAGE         STATUS          PORTS
web       nginx:1.27    Up 3 seconds    0.0.0.0:8080->80/tcp
```

그냥 docker ps 면 열이 더 많다. 화면이 좁아 골라 본 것이다.

멈춘 것까지 전부

```
$ docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES     IMAGE         STATUS
once      hello-world   Exited (0) 2 seconds ago
web       nginx:1.27    Up 3 seconds
```

once 는 할 일을 마치고 끝났지만 아직 남아 있다. ps 는 도는 것, ps -a 는 있는 것.

4. 컨테이너가 사는 세계

컨테이너의 안과 밖



포트는 이어 주어야 열리고, 데이터는 밖에 두어야 남는다.

포트

이어 주지 않으면 닫혀 있습니다

안에서 80번을 열어도, 그 80번은 컨테이너 안의 80번이기 때문입니다

컨테이너마다 자기 네트워크가 있어

여럿이 모두 80번을 써도 충돌하지 않습니다 — 2장의 포트 문제가 이렇게 풀립니다

데이터

그냥 두면 사라집니다

DB를 컨테이너로 띄우고 자료를 쌓았다면, 지우는 순간 자료도 같이 사라집니다

그래서 밖에 둡니다

볼륨

Docker가 관리하는 저장 공간
컨테이너를 지워도 남는다

운영에서 쓰는 방식

바인드 마운트

내 컴퓨터 폴더를 그대로 붙인다
고치는 즉시 안에도 반영된다

개발할 때 쓰는 방식

둘 다 “컨테이너 밖에 두는 것” 이다. 누가 관리하느냐가 다르다.

설정

이미지에 굽지 말고
넣어 줍니다

환경 변수로 넣으면 같은 이미지가 개발에서도 운영에서도 그대로 쓰입니다

비밀값은 특히

구우면 받은 사람이 레이어를 뒤져 꺼낼 수 있고, 지워도 기록에 남습니다

컨테이너는
오래 모시는 것이 아닙니다

고장 나면 고치는 대신 버리고 새로 띄웁니다

이제 직접 띄웁니다

nginx 를 8080 으로

```
$ docker run -d -p 8080:80 --name web nginx:1.27
c6e8139bc0550444c58f79e7b3413136cab879409827bd63bc06313f1f6d9d98

# -d        백그라운드로 띄운다
# -p        호스트 8080 → 컨테이너 80
# --name    이름을 붙인다 (안 주면 랜덤한 이름이 붙는다)
```

돌려주는 긴 문자열이 컨테이너 ID다. 앞 열두 자리만 써도 된다.

브라우저에서 localhost:8080

nginx 기본 화면이 뜨면 성공입니다

`docker logs -f web`

이 컨테이너가 뱉는 것을 본다

-f 는 계속 따라가기. Ctrl+C 로 빠져나옵니다

두 번 접속해 보면

```
$ docker logs web
2026/09/26 17:06:41 [notice] 1#1: nginx/1.27.5
2026/09/26 17:06:41 [notice] 1#1: start worker processes
192.168.65.1 - - [26/Sep/2026:17:06:44 +0000] "GET / HTTP/1.1" 200 615
192.168.65.1 - - [26/Sep/2026:17:06:44 +0000] "GET /nope HTTP/1.1" 404 153
```

로그가 파일이 아니라 화면으로 나온다. 9장의 “로그는 표준 출력으로” 가 이 이야기다.

`docker exec -it web sh`

돌고 있는 컨테이너 안으로 들어간다

-it 는 키보드를 연결하라는 뜻. exit 로 나옵니다

안에서 확인해 보면

```
$ docker exec web nginx -v
nginx version: nginx/1.27.5
```

안에 들어가 보면 작은 리눅스처럼 보인다. 하지만 호스트에서는 프로세스 하나다.

볼륨 — 내 폴더를 붙인다

```
docker run -d -p 8080:80 \
  -v $(pwd)/html:/usr/share/nginx/html:ro \
  --name web nginx:1.27

# -v  호스트 경로 : 컨테이너 경로 : ro(읽기 전용)
```

호스트에서 파일만 고치면

```
$ curl -s http://localhost:8080/
<h1>Hello from my folder</h1>

# html/index.html 을 고치고 다시
$ curl -s http://localhost:8080/
<h1>Changed without rebuilding</h1>
```

다시 빌드하지 않았는데 바뀐다. 개발할 때 바인드 마운트를 쓰는 이유다.

환경 변수는 -e 로 넣는다

```
$ docker run --rm -e GREETING=hello-swe nginx:1.27 sh -c 'echo $GREETING'
hello-swe
```

--rm 은 끝나면 컨테이너를 알아서 치우라는 뜻이다.

치울 때는 두 줄

```
$ docker stop web
web
$ docker rm web
web
$ docker ps -a --filter name=^web$
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

rm 하면 쓰기 레이어가 사라진다. 볼륨에 둔 것만 남는다.

5. Dockerfile

설치 과정을
글로 적습니다

기계가 실행해서 이미지를 만듭니다 — 문서와 현실이 어긋날 수 없습니다

다섯 줄이면 대충 이렇게 생겼습니다

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

한 줄씩 무슨 뜻인지만 알면 됩니다

FROM — 어떤 베이스에서 시작할지

COPY — 내 파일을 이미지 안으로 넣는다

RUN — 이미지를 만드는 동안 실행할 명령

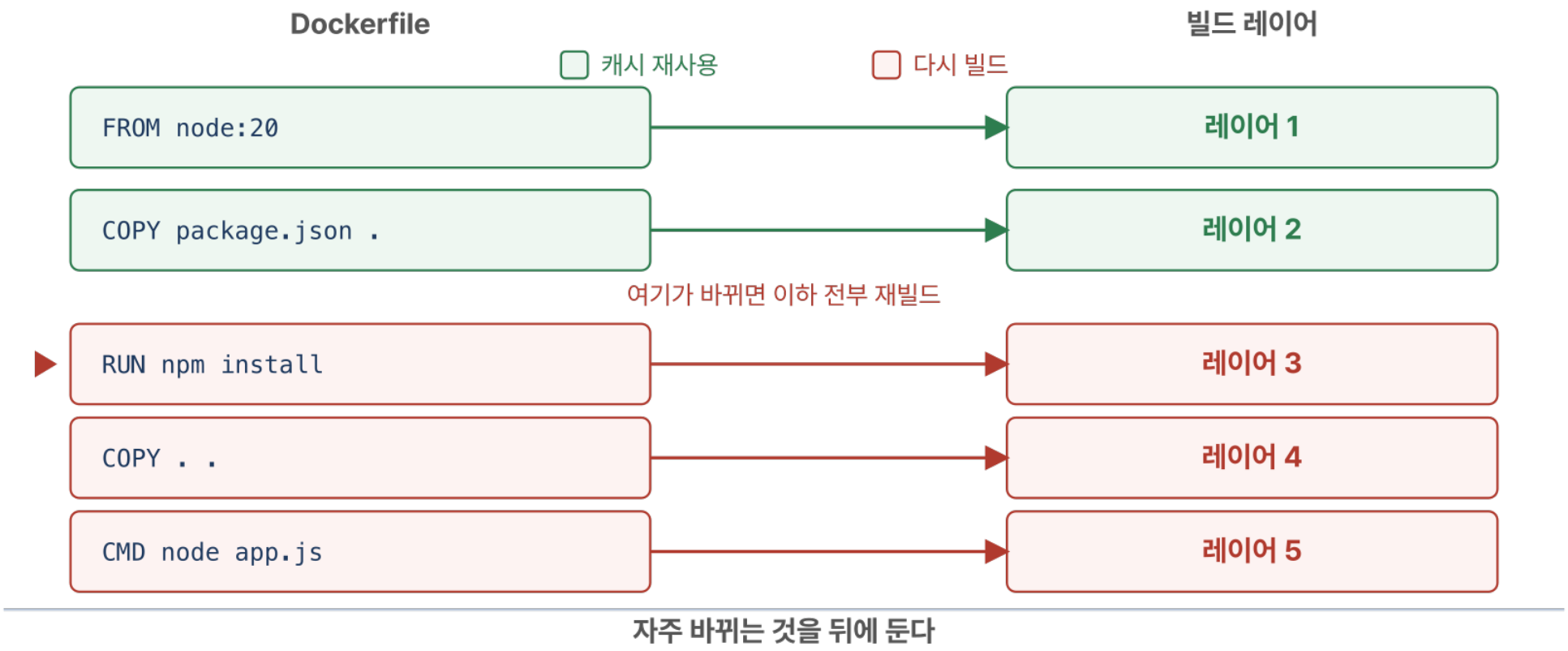
CMD — 컨테이너가 시작될 때 실행할 명령

RUN 은 만들 때 한 번
CMD 는 띄울 때마다

빌드 캐시 — 줄 하나가 레이어 하나

바뀐 게 없으면 이전 결과를 그대로 쓰고, 바뀐 줄부터 아래는 전부 다시 만듭니다

캐시가 깨지는 지점



한 줄이 바뀌면 그 아래는 모두 다시 만들어진다.

그래서
자주 바뀌는 것을 뒤에 둡니다

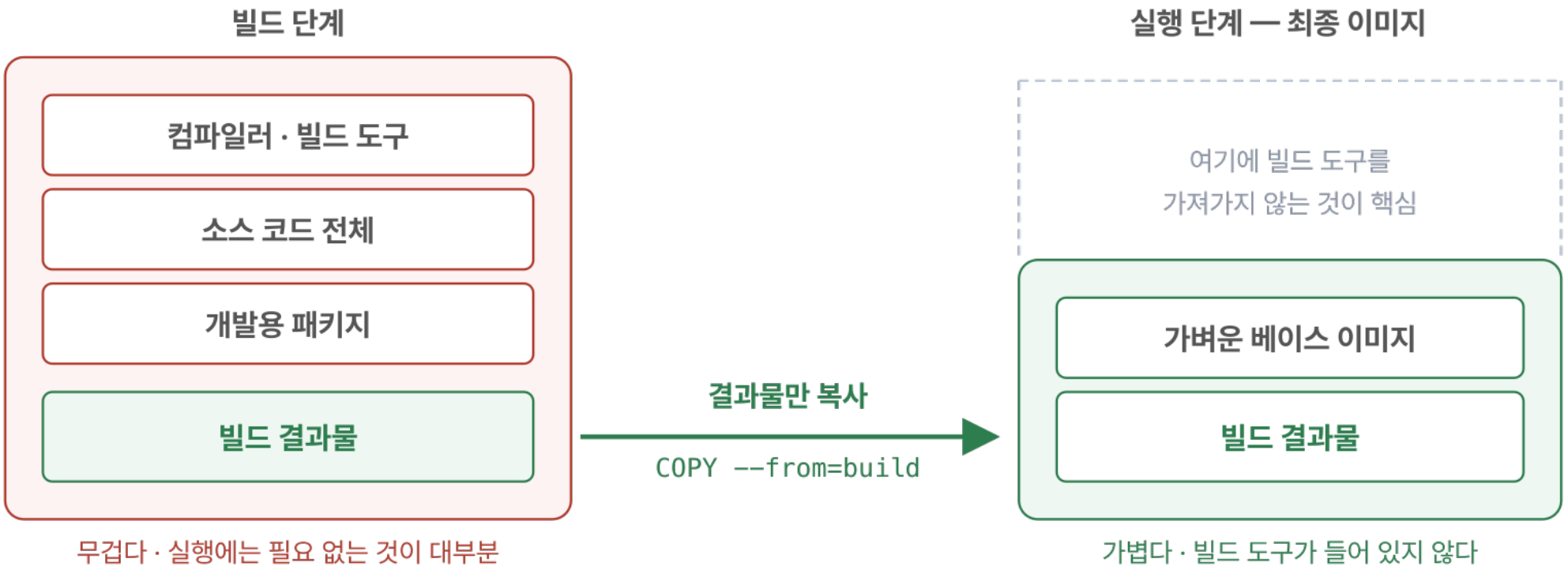
코드는 하루에도 수십 번 바뀌고, 의존성 목록은 며칠에 한 번 바뀝니다

.dockerignore

모아 가는 폴더가 빌드 컨텍스트입니다. .gitignore 와 같은 자리의 장치입니다

멀티스테이지

만드는 도구는 두고 결과만



실행용 이미지에는 결과물만 복사해 넣는다. 이미지가 몇 배 작아진다.

AI에게 시킬 때 확인할 세 가지

베이스 이미지 태그가 고정되어 있는가 (latest 금지)

비밀번호나 키가 파일 안에 적혀 있지 않은가

자주 바뀌는 코드 복사가 의존성 설치보다 뒤에 있는가

이 셋은 AI가 자주 빠뜨립니다

무엇을 확인해야 하는지 모르면 그대로 쓰게 됩니다

직접 만들어 봅니다

내 앱 — app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
    return "Hello, Software Engineering!\n"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

8000 번으로 한 줄 돌려주는 앱. host 를 0.0.0.0 으로 둬야 컨테이너 밖에서 닿는다.

requirements.txt 와 Dockerfile

```
# requirements.txt
flask==3.0.3

# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

노트 5장의 다섯 줄 그대로다.

```
docker build -t myapp:1.0
.
```

Dockerfile 로 이미지를 만든다

-t 는 붙일 이름표, 맨 끝의 점은 빌드 컨텍스트입니다

빌드 출력 (줄임)

```
$ docker build -t myapp:1.0 .
#6  [1/5] FROM docker.io/library/python:3.12-slim@sha256:f77ac9e4...
#7  [2/5] WORKDIR /app
#8  [3/5] COPY requirements.txt .
#9  [4/5] RUN pip install -r requirements.txt
#10 [5/5] COPY . .
#11 naming to docker.io/library/myapp:1.0 done

# 10.195s 걸렸다
```

Dockerfile 다섯 줄이 [1/5] ~ [5/5] 다섯 단계로 그대로 실행된다.

띄우고 열어 보면

```
$ docker run -d -p 8080:8000 --name myapp myapp:1.0
7661a3c465d7fe0512908177cfdb7f79b451679712e97148e5caab5e49b853aa
$ curl -s http://localhost:8080/
Hello, Software Engineering!
```

컨테이너 안 8000 을 호스트 8080 으로 이어 줬다.

코드 한 줄 고치고 다시 빌드

```
$ docker build -t myapp:1.1 .
#6 [3/5] COPY requirements.txt .
#6 CACHED
#7 [2/5] WORKDIR /app
#7 CACHED
#8 [4/5] RUN pip install -r requirements.txt
#8 CACHED
#9 [5/5] COPY . .
#9 DONE 0.1s

# 1.001s
```

pip install 이 CACHED 로 건너뛴다. 바뀐 것은 맨 아래 COPY 뿐이다.

10.195초가

1.001초가 됐습니다

이번엔 순서를 뒤집어 봅니다

```
# Dockerfile.bad
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

COPY . . 를 pip install 앞으로 옮겼다. 그것 하나만 다르다.

같은 수정, 같은 재빌드

```
$ docker build -f Dockerfile.bad -t badapp:1.1 .
#6 [2/4] WORKDIR /app
#6 CACHED
#7 [3/4] COPY . .
#8 [4/4] RUN pip install -r requirements.txt
#8 1.250 Collecting flask==3.0.3 (from -r requirements.txt (line 1))
#8 2.136 Successfully installed flask-3.0.3 Jinja2-3.1.6 ...

# 4.116s
```

코드 한 줄 고쳤는데 flask 를 다시 받아 온다. 캐시가 거기서 깨졌기 때문이다.

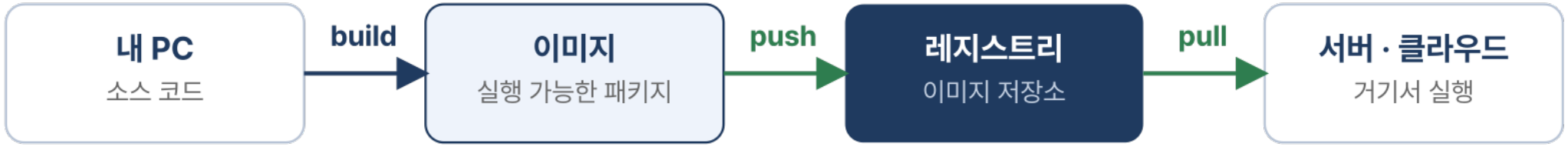
1.001초 대 4.116초

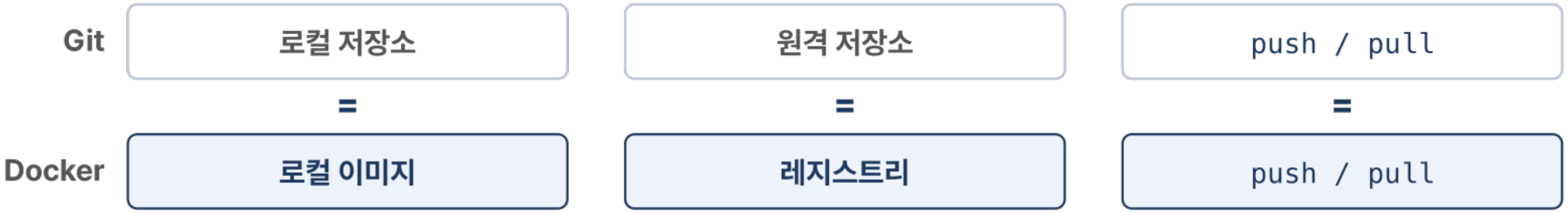
예제가 작아서 4배입니다. 실무에서는 10초와 5분으로 갈립니다

6. 레지스트리

만든 곳과 실행할 곳이 다르다
— 그게 Docker의 쓸모입니다

빌드는 한 번, 실행은 여러 곳





내 컴퓨터에서 빌드해 푸시하면, 서버나 클라우드가 풀해서 띄운다.

docker run nginx 가 그냥 되는 이유

내 컴퓨터에 없으면 Docker Hub에서 자동으로 받아 오기 때문입니다

Git과 Docker의 대응

Git (지난 시간)		Docker (이번 시간)
커밋 해시	=	이미지 다이제스트
태그	=	태그
원격 저장소	=	레지스트리
`push / pull`	=	`push / pull`
`.gitignore`	=	`.dockerignore`
스냅샷	=	레이어

지난 시간에 배운 구조가 이름만 바뀌어 그대로 반복된다.

다른 점도 하나 있습니다

Git은 소스 코드를, 레지스트리는 실행 가능한 결과물을 주고받습니다

latest 를 믿지 마라

“가장 최신” 이 아닙니다

그냥 기본값 이름표라서, 누가 같은 이름으로 올리면 그쪽으로 옮겨 갑니다

myapp:1.4.2

버전을 박아서 붙인다

"바꾼 게 없는데 어제 되던 게 안 된다" — 그 사고는 대부분 태그에서 납니다

태그를 하나 더 붙여 보면

```
$ docker tag myapp:1.0 blissray/myapp:1.0
$ docker images --filter reference='myapp' --filter reference='*/myapp'
IMAGE                  ID             DISK USAGE   CONTENT SIZE
blissray/myapp:1.0     ce38e4567f19         225MB         49.4MB
myapp:1.0              ce38e4567f19         225MB         49.4MB
```

ID가 같다. 복사한 게 아니라 같은 이미지에 이름표만 하나 더 붙인 것이다.

올리고 내리는 건 세 줄

```
docker login
docker push blissray/myapp:1.0
docker pull blissray/myapp:1.0

# 이름 앞에 계정 이름이 붙어야 내 저장소로 올라간다
```

push 는 오늘 하지 않습니다

Docker Hub 계정이 필요합니다. 흐름만 눈에 넣어 두세요

7. 여러 개를 한꺼번에

앱 하나만 도는 경우는 거의 없습니다

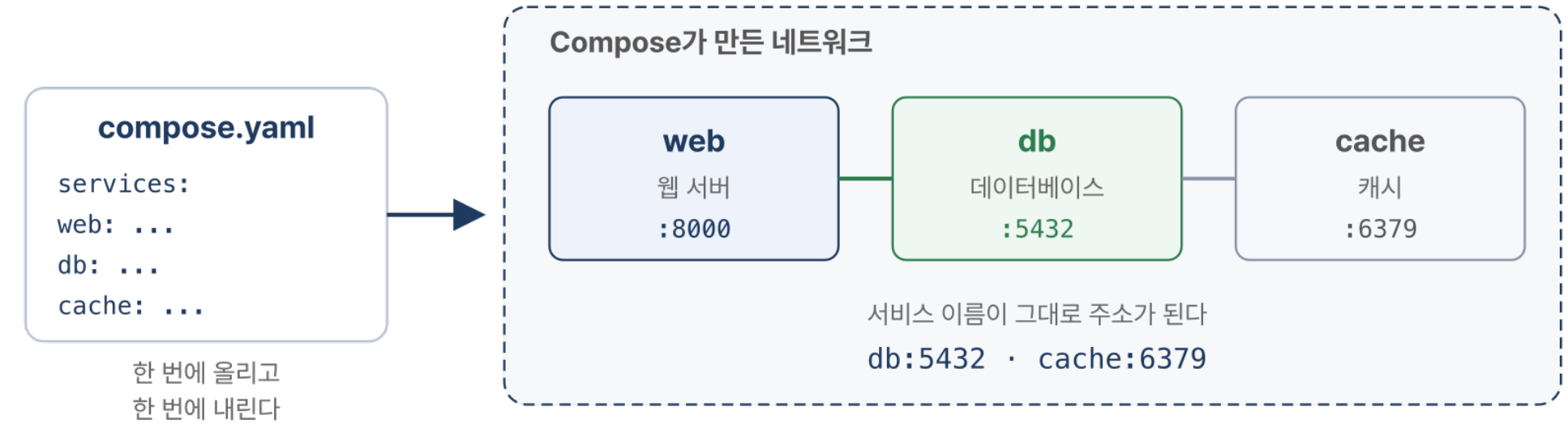
웹 서버, 데이터베이스, 캐시 — 컨테이너 세 개입니다

매번 손으로 띄우려면

옵션이 줄줄이 붙은 명령을 세 번, 순서도 맞춰야 합니다. 매일은 못 합니다

Compose는 이 구성을 파일 하나에 적어 둡니다

compose.yaml 하나로



`localhost:5432`
컨테이너 안에서는 자기 자신이다

`db:5432`
같은 네트워크의 서비스 이름으로 부른다

새 기술이 아니라, 여러 컨테이너를 한 벌로 묶어 선언해 두는 방법이다.

반드시 한 번 막히는 지점

주소를 localhost 로 적습니다

안 됩니다. 컨테이너 안에서 localhost 는 자기 자신을 가리킵니다

db:5432

서비스 이름이 그대로 주소

함께 띄운 컨테이너들은 같은 네트워크에 들어갑니다

compose.yaml 은 이 정도

```
services:
  web:
    build: .
    ports:
      - "8080:8000"
    environment:
      - CACHE_HOST=cache
    depends_on:
      - cache
  cache:
    image: redis:7-alpine
```

서비스 둘. web 은 내 Dockerfile 로 빌드하고, cache 는 이미지를 받아서 쓴다.

`docker compose up -d`

한 번에 올린다

빌드가 필요하면 빌드까지 하고, 네트워크도 만들어 줍니다

올려 보면

```
$ docker compose -p swe-demo up -d
 Network swe-demo_default  Created
 Container swe-demo-cache-1  Started
 Container swe-demo-web-1  Started

$ docker compose -p swe-demo ps
NAME                IMAGE               SERVICE   STATUS          PORTS
swe-demo-cache-1    redis:7-alpine      cache     Up 4 seconds    6379/tcp
swe-demo-web-1      swe-demo-web        web       Up 4 seconds    ...->8000/tcp
```

-p 는 프로젝트 이름이다. 컨테이너 이름이 <프로젝트>-<서비스>-<번호> 로 붙는다. (열 줄임)

이름으로 정말 찾아지는지

```
$ docker compose -p swe-demo exec -T web \
    python -c "import socket; print('cache ->', socket.gethostbyname('cache'))"
cache -> 192.168.208.2
```

localhost 가 아니라 서비스 이름 cache 가 주소로 풀린다. 앞에서 말한 규칙 그대로다.

내릴 때는 한 줄

```
$ docker compose -p swe-demo down
 Container swe-demo-web-1  Removed
 Container swe-demo-cache-1  Removed
 Network swe-demo_default  Removed
```

컨테이너도 네트워크도 같이 치운다. 올릴 때 만든 것을 내릴 때 되돌린다.

쿠버네티스는 무엇인가

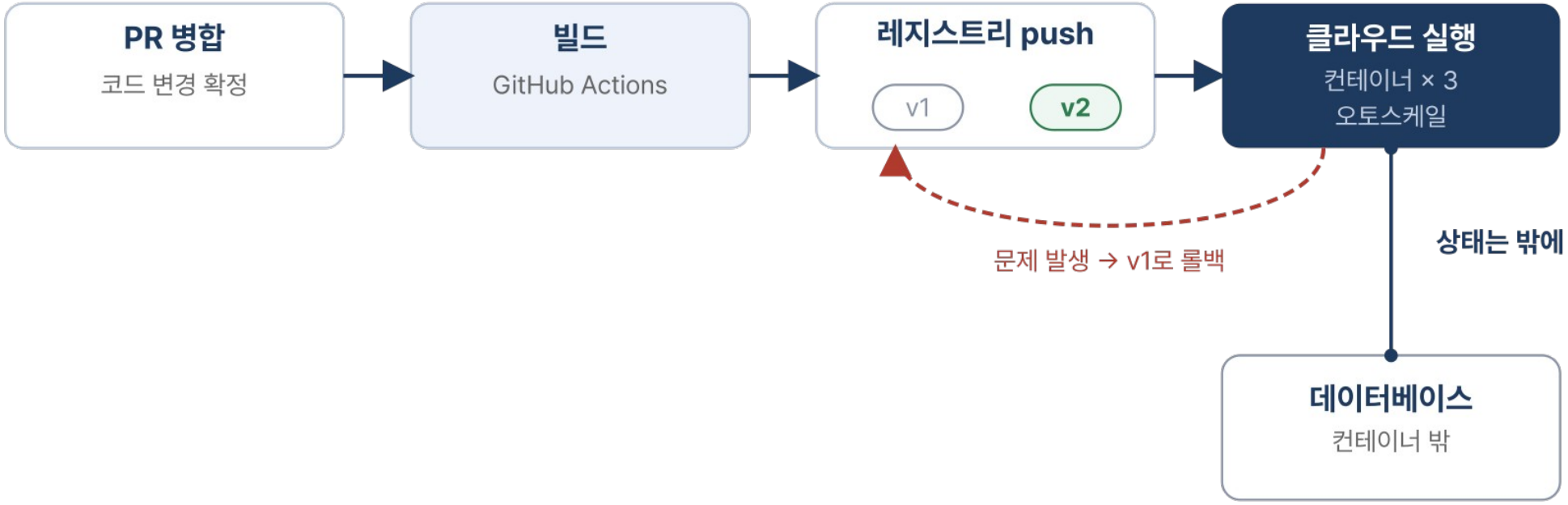
Compose가 한 대라면, 쿠버네티스는 여러 대에 걸쳐 나눠 띄우고 관리합니다

8. 클라우드에서 왜 중요한가

여기까지가
내 컴퓨터 안의 이야기였습니다

이 장이 “이걸 왜 배우나” 에 대한 답입니다

PR 병합에서 클라우드까지



컨테이너는 사라져도 됨 · 데이터는 DB에 남는다

PR이 병합되면 이미지가 만들어지고, 클라우드는 그 이미지를 띄운다.

① 예전 배포는 이런 일이었습니다

서버에 접속한다

코드를 내려받는다

필요한 것을 설치한다

설정 파일을 고친다

서비스를 재시작한다

이제 배포는 이미지를 올리고
띄우라고 말하는 일입니다

서버엔 아무것도 설치하지 않고, 테스트한 그 이미지가 그대로 올라갑니다

② 레지스트리가 중심이 됩니다

Docker Hub 자리에 클라우드의 레지스트리가 들어갑니다 — AWS라면 ECR

③ 클라우드가 파는 것

“대신 돌려 주는 일”

서버를 빌린다

AWS EC2
켜고, Docker를 깔고, 띄우고,
서버를 계속 관리한다

자유롭지만 손이 많이 간다

이미지만 맡긴다

ECS · Fargate, App Runner,
Cloud Run, Container Apps
이미지 주소와 개수만 알려 준다

편하지만 틀 안에서만 쓴다

학생 프로젝트라면 오른쪽. Fly.io · Render · Cloudtype 도 Dockerfile 하나면 띄워 준다.

어느 쪽을 고르든
만드는 것은 이미지 하나입니다

④ 롤백이
태그를 바꾸는 일이 됩니다

이전 태그의 이미지를 다시 띄우면 끝

몇 초입니다. 단, 전부 latest 하나로 덮어썼다면 돌아갈 지점이 없습니다

⑤ 같은 이미지로 여러 개 띄웁니다

이미지가 동일하니 몇 개를 띄우든 동작이 같습니다 — 오토스케일

문제는 데이터입니다

업로드 파일을 컨테이너 안에 저장하면, 다음 요청은 그 자료를 못 찾습니다

그래서 상태는
컨테이너 밖에 둡니다

⑥ 병합하면 자동으로 이어집니다

GitHub Actions 가 이미지를 만들고

레지스트리에 올리고

클라우드가 새 이미지로 갈아 끼운다

⑦ 이미지 크기는
시간이고 돈입니다

급히 늘려야 할 때 내려받는 데 1분이 걸리면, 그 1분 동안 사용자는 기다립니다

9. 실무의 약속들

① 이미지는 작게

가벼운 베이스, 멀티스테이지로 빌드 도구 떼기, .dockerignore

② 태그는 고정

latest 대신 버전을 박습니다. 되돌릴 지점을 남기는 일입니다

③ 한 컨테이너에 한 가지 일

따로 두어야 각각 늘리고 줄일 수 있고, 하나가 죽어도 다른 하나가 삽니다

④ 상태는 밖에

남아야 하는 데이터는 볼륨이나 외부 저장소에 둡니다

⑤ 로그는 표준 출력으로

안의 파일에 쌓으면 컨테이너와 함께 사라집니다

⑥ 비밀값은 이미지 밖에

환경 변수나 비밀 저장소로 주입합니다

⑦ 빌드는 재현 가능하게

버전을 고정하지 않으면 같은 Dockerfile로 어제와 다른 이미지가 나옵니다

외울 것은 일곱 개가 아니라

각각의 “왜” 입니다

그리고 하나 더 — 청소

`docker system df`

무엇이 자리를 얼마나 먹고 있나

이미지 · 컨테이너 · 볼륨 · 빌드 캐시 네 줄로 보여 줍니다

실제로 재 보면

```
$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          128       35        38.3GB    1.238GB (3%)
Containers      44        28        472.7MB   128.3MB (27%)
Local Volumes   215       28        24.51GB   12.71GB (51%)
Build Cache     509       0         20.57GB   783.4MB
```

교수 노트북에서 실제로 잰 값이다. 쓰다 보면 금세 이렇게 된다.

`docker system prune`

안 쓰는 것을 지운다

멈춘 컨테이너 · 안 쓰는 네트워크 · 떠도는 이미지를 지웁니다

지우기 전에 목록을 읽으세요

무엇을 지울지 먼저 보여 주고 y 를 묻습니다. 남의 컴퓨터에서는 특히

10. 이 수업에서 쓰는 방식

제출물은 배포된 URL입니다

순서가 중요합니다

① **Dockerfile을 하나 쓴다**

② **로컬에서 컨테이너로 띄워 본다**

③ **그다음 클라우드에 올린다**

로컬 확인을 건너뛰면 원인이
내 코드인지 클라우드 설정인지 가릴 수 없습니다

AI에게 맡기되

그대로 쓰기 전에 네 가지만

베이스 이미지 태그가 고정되어 있는가

비밀값이 파일에 박혀 있지 않은가

데이터가 컨테이너 안에만 저장되고 있지 않은가

내 컴퓨터에서 컨테이너로 떠서 브라우저로 열리는가

11. 이제 직접 해 봅니다

오늘 따라 할 순서

① 설치 확인 ② hello-world ③ nginx 띄워 브라우저로 열기

④ 로그 · 안쪽 보기 ⑤ 치우기

⑥ 내 앱에 Dockerfile ⑦ 빌드 ⑧ 띄우고 확인 ⑨ 치우기

막히면 손 드세요

넘어가도 됩니다. 오늘 못 하면 다음 주에 다시 합니다

① 설치 확인 — 둘 다 나오면 준비 끝

```
$ docker version
Client:  Version:  29.8.0
Server:  Docker Desktop
```

Client 만 나오고 Server 가 없으면 Docker Desktop 이 안 떠 있는 것이다.

Server 가 안 나오면

Docker Desktop 을 먼저 켜세요. 고래 아이콘이 멈춰 있으면 아직 준비가 안 된 겁니다

② hello-world

```
$ docker run hello-world

# 기대 — Hello from Docker! 가 보이면 성공
```

이 한 줄이 되면 받기 · 만들기 · 실행하기가 전부 동작한다는 뜻이다.

③ nginx 띄우기

```
$ docker run -d -p 8080:80 --name web nginx:1.27

# 기대 — 64자리 컨테이너 ID 한 줄
```

그다음 브라우저에서 localhost:8080 을 연다.

브라우저에서 localhost:8080

“Welcome to nginx!” 가 보이면 성공입니다

안 열리면 -p 를 확인하세요

-p 없이 띄웠으면 컨테이너 안에서만 열려 있습니다

④ 로그와 안쪽 보기

```
$ docker logs web
$ docker exec -it web sh

# 기대 — 접속 기록 한 줄, 그리고 컨테이너 안 쉘 프롬프트
# 나올 때는 exit
```

⑤ 치우기

```
$ docker stop web
$ docker rm web

# 기대 – 두 번 다 web 이라고 한 줄씩 돌려준다
```

여기까지가 남의 이미지

이제 내 것을 만듭니다

⑥ 내 앱에 Dockerfile 한 장

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Node 를 쓰면 FROM 만 바뀝니다

node:20-slim 에 npm install, 나머지 구조는 같습니다

AI에게 시켜도 됩니다

대신 아까 그 세 가지는 직접 확인하세요 — 태그 고정 · 비밀값 · 복사 순서

⑦ 빌드

```
$ docker build -t myapp:1.0 .

# 기대 – 끝에 naming to docker.io/library/myapp:1.0 done
```

한 번 더 쳐 보면 CACHED 가 줄줄이 뜨고 훨씬 빨리 끝난다.

⑧ 띄우고 확인

```
$ docker run -d -p 8080:8000 --name myapp myapp:1.0
$ curl -s http://localhost:8080/

# 기대 — 내 앱이 돌려주는 문자열
```

브라우저로 열어도 된다. 여기까지 오면 오늘 목표 달성이다.

⑨ 치우기

```
$ docker stop myapp
$ docker rm myapp
$ docker system df

# 마지막 줄은 무엇이 얼마나 쌓였는지 보는 것
```

막히는 자리 셋

① 포트가 이미 쓰이고 있을 때

```
$ docker run -d -p 8080:80 --name web2 nginx:1.27
docker: Error response from daemon: failed to set up container
networking: ... Bind for 0.0.0.0:8080 failed: port is already allocated
```

해결 — 다른 포트를 쓰거나(-p 8081:80), 그 포트를 쓰던 컨테이너를 stop 한다. (한 줄을 접었다)

② 이름이 이미 있을 때

```
$ docker run -d -p 8081:80 --name web nginx:1.27
docker: Error response from daemon: Conflict. The container name
"/web" is already in use by container "c6e8139bc055...". You have to
remove (or rename) that container to be able to reuse that name.
```

해결 — docker rm web 으로 지우고 다시 띄우거나, 다른 이름을 준다. (한 줄을 접었다)

③ Cannot connect to the Docker daemon

Docker Desktop 이 떠 있는지부터 봅니다. 대부분 이것입니다

에러 메시지를 그대로 읽으세요

Docker 의 오류는 무엇이 문제인지 거의 그대로 적어 줍니다

한 장으로 줄이면

오늘 쓴 명령어 전부

이미지 다루기

```
docker pull nginx:1.27
docker images
docker tag myapp:1.0 me/myapp:1.0
docker push me/myapp:1.0
```

컨테이너 다루기

```
docker run -d -p 8080:80 nginx:1.27
docker ps   /   docker ps -a
docker logs -f web
docker exec -it web sh
```

만들기

```
docker build -t myapp:1.0 .
docker run -p 8080:8000 myapp:1.0
docker compose up -d
docker compose down
```

치우기

```
docker stop web  /  docker rm web
docker rmi myapp:1.0
docker system df
docker system prune
```

외우지 마세요. 필요할 때 이 장을 찾아보거나, AI에게 시키고 결과를 읽으면 됩니다.

다시 말하지만

시험에는 명령어가 나오지 않습니다

오늘 손으로 해 본 것은

개념이 진짜인지 눈으로 확인하려고 한 것입니다

다음으로

프로젝트가 모양을 갖추면

Dockerfile을 쓰고, 로컬 컨테이너로 확인하고, 클라우드에 올립니다

END

감사합니다